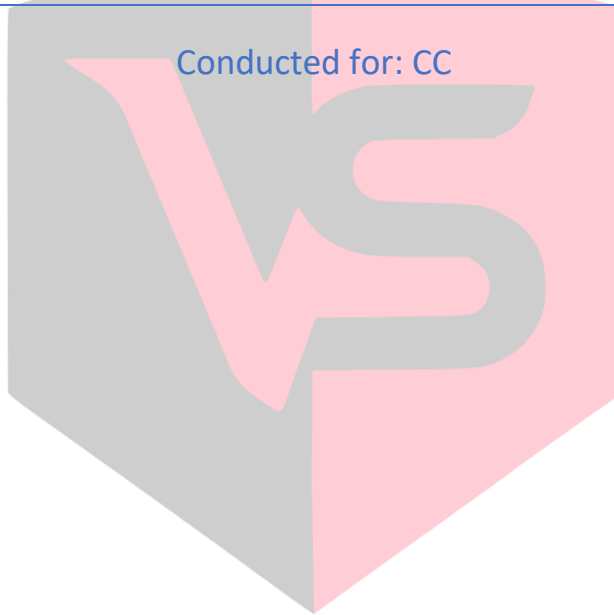




SMART CONTRACT AUDIT

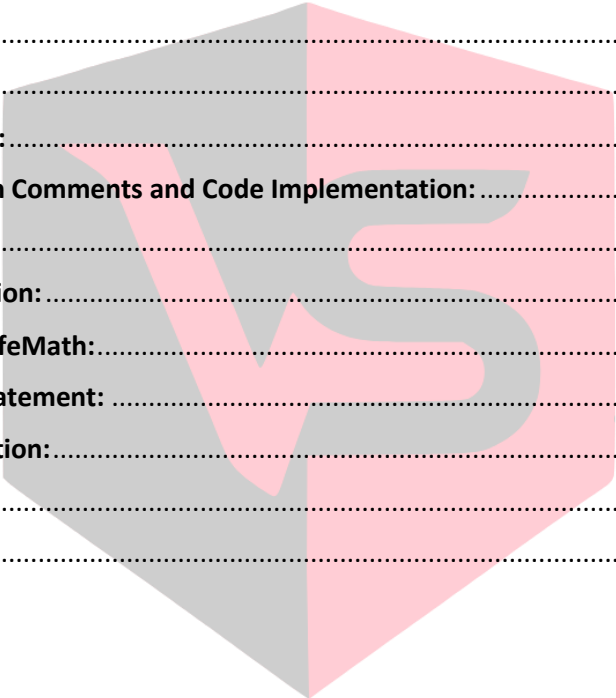
Conducted for: CC



VULSIGHT

Contents

Executive Summary:	3
Project Background:	3
Audit Scope:	3
Audit Summary:	3
Severity Definitions:	4
Technical Checks:	4
Code Quality and Documentation:	5
Audit Findings:	5
Critical vulnerabilities:	5
High vulnerabilities:	5
Medium vulnerabilities:	5
Discrepancy between Comments and Code Implementation:	5
Low vulnerabilities:	6
Lacks Of Event Emission:	6
Redundant Use of SafeMath:	6
Misplaced require statement:	7
Unused Private Function:	7
Disclaimer:	9
Conclusion:	9



VULSIGHT

Executive Summary:

The client contacted our Firm to conduct a smart contract audit on their Solidity based smart contract. The smart contract was deployed on the Ethereum chain. The provided smart contract appears to be an extended version of the ERC-20 token standard, incorporating advanced functionalities tailored to modern DeFi applications. At its core, the contract enables the creation, transfer, and management of a digital token named "CryptoCoin". The audit was performed using both manual code analysis and automated security tools. The audit is performed on a best effort basis. Few vulnerabilities of low and medium severity were detected in the audit. No critical vulnerabilities were detected within the smart contract code during the course of this audit. All of the findings of the audit are detailed inside this report.

Project Background:

The smart contract is written in solidity. This token includes owner functionality. It is deployed on the Ethereum mainnet.

Audit Scope:

Deployed Address	0x28acfe1bd2f4ebb781582d89c7616d713d4e4ec0
Chain	Ethereum MainNet
Language	Solidity
Audit Completion Date	16/8/23

Audit Summary:

- **Fee Handling:** The contract has a built-in mechanism to apply fees on token transfers. These fees are adjustable and can vary depending on whether a transaction is a purchase or a sale. There are also provisions to exempt certain addresses from these fees.
- **Liquidity Management:** A distinct feature is the automatic conversion of tokens into Ether, presumably to maintain liquidity in decentralized exchanges. This is managed through integration with the Uniswap protocol.
- **Transfer and Wallet Limits:** The contract allows for restrictions on the maximum amount of tokens that can be transferred in a single transaction. It also provides a mechanism to limit the maximum amount of tokens an address can hold, with exceptions available.

- **Central Management Tools:** The owner possesses significant control over various parameters of the contract, including toggling certain features, rescuing stuck tokens, and manually adjusting balances for airdrop campaigns.

We found:

- **0 Critical risk vulnerabilities**
- **0 High risk vulnerabilities**
- **1 Medium risk vulnerability**
- **4 Low risk vulnerabilities**

Severity Definitions:

Risk Level	Description
Critical	Any kind of vulnerability that could lead to direct token or monetary loss
High	Any type of vulnerability that could disrupt the proper functioning of smart contract or indirectly lead to token or monetary loss.
Medium	Any type of vulnerability that could cause undesired actions but no serious disruption or monetary loss
Low	Any type of vulnerability that doesn't have any significant impact on the execution of the proper functioning of contract

Technical Checks:

Checks	Result
Solidity version not specified	Passed
Solidity version too old	Passed
Integer overflow/underflow	Passed
Function input parameters check bypass	Passed
Insufficient randomness used	Passed
Fallback function misuse	Passed
Race Condition	Passed
Logical Vulnerabilities	Not Passed
Function visibility not explicitly declared	Passed
Use of deprecated keywords/functions	Passed
Out of Gas issue	Passed
Front Running	Passed
Insufficient Decentralization	Passed
Function input parameters lack of check	Passed
Proper function Access Control	Passed
Hardcoded Data	Passed

Code Quality and Documentation:

The audit scope included one smart contract which was written in Solidity programming language. The code is well indented and clear in nature. While the Code lacked commenting.

Audit Findings:

Critical vulnerabilities:

0 critical vulnerabilities were found in the smart contract.

High vulnerabilities:

0 high vulnerabilities were found in the smart contract.

Medium vulnerabilities:

1 medium vulnerability was found in the smart contract.

Discrepancy between Comments and Code Implementation:

In the contract, the functions **setBuyTaxes** and **setSellTaxes** are responsible for setting the buy and sell tax rates respectively. While the comment documentation of both functions suggests that they accept a maximum limit of 25%, the implemented code restricts the maximum limit to just 4%. This is evident from the statement `(feeUnits/25)`, which, considering `feeUnits` likely represents a percentage base of 100, results in a maximum allowable tax of 4%.

```
function setBuyTaxes(uint _Marketing) public onlyOwner {  
  
    _buyMarketingFee = _Marketing;  
  
    _totalTaxIfBuying = _buyMarketingFee;  
    require(_totalTaxIfBuying <= (feeUnits/25), "Buy fees must be 25% or  
less");  
}  
  
function setSellTaxes(uint _Marketing) public onlyOwner {  
  
    _sellMarketingFee = _Marketing;  
  
    _totalTaxIfSelling = _sellMarketingFee;  
    require(_totalTaxIfSelling <= (feeUnits/25), "Sell fees must be 25% or  
less");  
}
```

It has the following implication:

- **Misleading Documentation:** Developers or users referencing the comment might assume that they can set the buy or sell tax rates up to 25%, leading to potential misunderstandings or incorrect use of the contract.
- **Implementation and Intent Mismatch:** If the original intent was to allow up to 25% for the tax rates, the current code does not align with that purpose.
- **Possible Incorrect Contract Use:** Owners might attempt to set tax rates above 4% (but under 25%), thinking it's allowed, only to encounter transaction reversion, causing frustration and potential loss of gas.

Low vulnerabilities:

4 low vulnerabilities were found in the smart contract.

Lacks Of Event Emission:

Upon reviewing the provided contract, it was observed that several functions lack corresponding event emissions. Not having events in critical functions can make it challenging to track the actions of a contract, especially when dealing with complex dApps or when there's a need for off-chain systems to respond to on-chain changes. Crucial Functions that lack such events include:

- **rescueStuckedToken**
- **rescueFunds**
- **setWalletLimit**
- **setMaxTxAmount**
- **setBuyTaxes**
- **setSellTaxes**

Redundant Use of SafeMath:

In Solidity versions prior to 0.8, integer overflows and underflows were a notable concern. The SafeMath library was widely adopted by the community to handle these arithmetic anomalies by ensuring that operations like addition, subtraction, multiplication, and division would fail (revert) if they resulted in an overflow or underflow. This library added an extra layer of safety against unintentional errors in contract logic.

However, with the introduction of Solidity 0.8 and later versions, overflow and underflow checks were integrated directly into the language. This means that arithmetic operations in Solidity 0.8+ automatically revert on integer overflows or underflows, making the SafeMath library redundant and unnecessary.

From the provided contract, it's evident that SafeMath functions (**e.g., add, sub, mul, div**) are being used, even though they are not necessary with Solidity 0.8+. This not only adds to the gas costs but also makes the code less concise.

Misplaced require statement:

The placement of require statements within a function has a direct impact on the efficiency of gas consumption. In the provided contract's **setBuyTax** and **setSellTax** functions, the require statements are positioned at the end, after several state-changing operations. This is problematic from a gas efficiency perspective.

```
function setBuyTaxes(uint _Marketing) public onlyOwner {  
  
    _buyMarketingFee = _Marketing;  
  
    _totalTaxIfBuying = _buyMarketingFee;  
    require(_totalTaxIfBuying <= (feeUnits/25), "Buy fees must be 25% or  
less");  
}  
  
function setSellTaxes(uint _Marketing) public onlyOwner {  
  
    _sellMarketingFee = _Marketing;  
  
    _totalTaxIfSelling = _sellMarketingFee;  
    require(_totalTaxIfSelling <= (feeUnits/25), "Sell fees must be 25% or  
less");  
}
```

When a function is invoked on the Ethereum network, every operation consumes gas. If a require statement fails (i.e., its condition is not met) and causes the function to revert, all the gas consumed by the operations preceding the require statement is wasted since the state changes are not persisted.

By placing the require statement towards the end of the function, the contract unnecessarily spends gas on prior operations even if the conditions in the require are not satisfied. This is not only wasteful but also places an unnecessary financial burden on the user, as they will be charged for the gas consumed by the failed transaction.

Unused Private Function:

The contract contains an unused private function named **transferToAddressETH**. Given its private visibility, it can only be executed from within the contract. If it's not called anywhere within the contract, it's effectively dormant code. It could lead to the following problems:

- **Gas Inefficiency:** Although the function itself isn't executed, its presence increases the contract's deployment cost.

- Increased Complexity: Dormant code adds unnecessary complexity to the codebase. New developers or auditors reviewing the contract might spend time understanding this function, even if it has no current use.
- Potential Confusion: Unused functions might lead developers to believe there is a missing piece of functionality or that there might be a future intention for its use. It can also lead to misinterpretations about the contract's intended behavior.



Disclaimer:

The smart contract was tested on a best-effort basis. The smart contract was analyzed with the best security practices known at the time of writing this report. Due to the fact that the total number of test cases is unlimited and the fact that new vulnerabilities in technologies are discovered every day, the audit report makes no guarantees on the security of the contract. It is recommended to not just rely on this audit report alone. A bug bounty program for this smart contract may be created to identify any vulnerabilities within the future.

Conclusion:

The smart contract was tested extensively both automatically and manually. Few Vulnerabilities were discovered within it. It is recommended these vulnerabilities be resolved. It is also recommended that additional security measures may be undertaken to ensure future security such that of a creation of a bug bounty program.

It is also recommended to use a proxy mechanism in the smart contract to ensure that the smart contract code can be modified in the future.

